Foundations in Data Engineering

Data Models

Jana Giceva



Recap from last lecture

- We said that SQL is now a Turing complete language.
  - **Q: Which feature did we cover that makes SQL Turing complete?** Recursive

- Things one can do with SQL using that feature:
  - https://cedardb.com/blog/doomql/

Recap from last lecture

Use the HyPer web-interface with the English UniSchema and:

- **Q: Find all direct and indirect prerequisites of lecture 5259.**

- **Q: Find the size of the transitive closure (all direct and indirect dependencies) of the *require* table.**

Data Models overview

- A ***data model*** is a collection of concepts for describing the data in a dataset.

- **Types of data models**:
  - Relational ← most DBMSs
  - Key/Value ← simple applications / caching
  - Graph
  - Document / JSON / XML / Object ← NoSQL
  - Wide-Column / Column-family
  - Array (Vector, Matrix, Tensor) ← ML / Science
  - Hierarchical
  - Network
  - Semantic ← Obsolete / Legacy / Rare
  - Entity-Relationship

← focus for this week

Today’s agenda

- Data models
  - Key/Value stores
  - Document / JSON/XML/Object
  - Wide-Column / Column-family
  - Graph
  - Array (Vector, Matrix, Tensor)

Learning objectives

- **Know** the main data models and be able to **name** at least one representative of each one.
- **Explain** why NoSQL systems emerged and why many later added SQL interfaces and ACID support.
- **Describe** the CAP theorem and the difference between strong and eventual consistency.
- **Model** a small dataset in several models: as a JSON document, as a property graph in Cypher, as RDF triples and as relational node/edges tables.
- **Compare** data models on schema flexibility, query style, scaling approach and consistency guarantees.
- **Justify** the choice of a data model and system for a given scenario.

Key Value data model

Key Value data model (1990s)

- The simplest data model possible – global hash map
  - `(key, value)`
  - `Keys` are usually primitives (e.g., `ints`, `string`, raw bytes, etc.)
  - Values can be primitives (`ints`, `strings`) or complex (JSON, HTML, blob): opaque to the data store

The application interprets this data

- A **KV store** (KV DBMS) is a collection of data as an associative array that maps a key to a value.

- The KV DBMS is unaware of the content → it is up to the **application** to **maintain** the **schema and parse the value** into its corresponding parts.
  - Most KV DBMSs only provide `get / set / delete` operations on a single value.
  - Most KV stores are used for applications like caching or storing session data.

.

Key Value stores

- With their limited functionality, they are **easy to implement** and **easy to scale**.

- **Very simple API:**
  - GET – fetch value associated with key
  - SET – set value associated with key
  - DELETE – delete the key

- **Optional operations:**
  - MultiGet – retrieve multiple records at once (as long as they share the same major path components)
  - MultiPut – allows to load records into the store
  - Range queries – retrieves a set of records specified by an upper and/or lower bound on keys

    data must be sorted

- **Consistency model:**
  - Atomic puts (usually)
  - Cross-key operations – often eventual consistency.

Dealing with scale

- **Partition the key space across multiple machines**
  - Let's say hash partitioning
  - For `n` machines, store `key k` at machine `h(k) mod n`

- Okay… but add a new machine might be expensive to redefine in which machine is the data stored
  - **How do we know which physical machines to contact?**
  - **How do we add new machines to the cluster?**
  - **What happens if a machine fails?**

- We need more sophisticated support than a simple global hash table
  - We will cover these challenges and various techniques on how to deal with it later in class, when we talk about distribution.

Key Value stores

- **Where are they predominantly used?**
  - *e.g.*, **Distributed KV stores** – Memcached, Redis, Dynamo, Riak
    - Offer higher and more predictable performance compared to an RDBMS, in exchange for more limited functionality.
  - *e.g.*, **Embedded storage managers** – BerkeleyDB, LevelDB, RocksDB
    - One could use an *embedded KV store* as a storage manager for full-feature RDBMS
    - *e.g.*, Meta used the flexible API of MySQL to replace the default InnoDB with RocksDB.

KV stores – Discussion

- **Quick way for developers to store data**
  - *Excellent* for specific narrow use-cases, like caching data.
  - *But,* using KV store in a complex application that requires more than a binary relation is tricky
    - With multiple fields – one must have the *application parse record fields*
    - *No secondary indexes* to retrieve other fields by value
    - *Poor ad-hoc querying.*
    - Limited to *no consistency*
- Some systems began as KV store and then evolved into more feature-rich record stores
  - replacing the opaque value with a semi-structured one (e.g., a JSON document).
  - evolving into document store (discussed next)
    e.g., Dynamo → DynamoDB.

NoSQL
Document data model

NoSQL databases

Google, Facebook early 2000's wanted a more flexible database

- **Traditional RDBMSs were seen as:**
  - slow
  - complex
  - inflexible
  - not-suited for web-usage
  - Expensive

- There was a **push** for something **radically** *simpler*, *faster* and *cheaper*.

- Something that could handle, what was perceived at the time as key **challenges for existing RDBMSs**
  - Write volume and horizontal scaling
  - Ability to work with semi-structured web data (XML → JSON)
  - Evolving schemas
  - Latency (and JOIN explosion)

lose consistency for the sake of latency

Example story

- **Facebook in 2008:**
  - billions of friend edges
  - constant schema evolution
  - global reads/writes.

- It was very difficult to model it in 3rd normal form (3NF)
  - *Q: What's 3NF?*

- It needed a system that can **scale horizontally** across many servers
- **High availability** and **robustness** was critical.
- It had to deal with a **vast and non-uniform data** (text, images and videos).

user will accept a lack of consistency for the sake of speed

casandra built by facebook

The rise of NoSQL

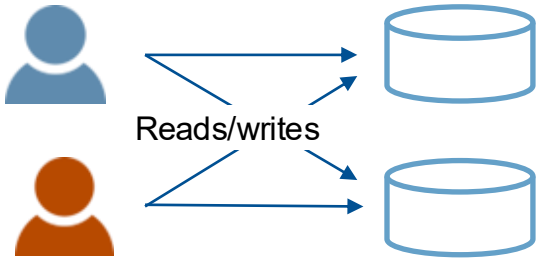
- The inability of OLTP RDBMSs to scale in the 2000s led to the rise of **NoSQL systems**.
  online transactional …
  - *Rationale 1:* SQL and joins are slow – one should use a "faster" lower-level, record-at-a-time interface
  - *Rationale 2:* ACID transactions are too heavy for modern applications, so DBMSs should only provide a weaker notion of it (e.g., BASE)
    - ACID – transactional integrity
    - BASE – Basically Available, Soft-state, Eventually consistent
  - *Rationale 3:* CAP theorem (Consistency / Availability / Partition tolerance)

- As a **result** NoSQL databases:
  - Store **records or documents as JSON**/BSON → allowing flexible schema
  - Are **supported by lower-level API** → no overhead of SQL and joins
  - Provide **limited consistency** → no ACID transactions to slow us down at scale

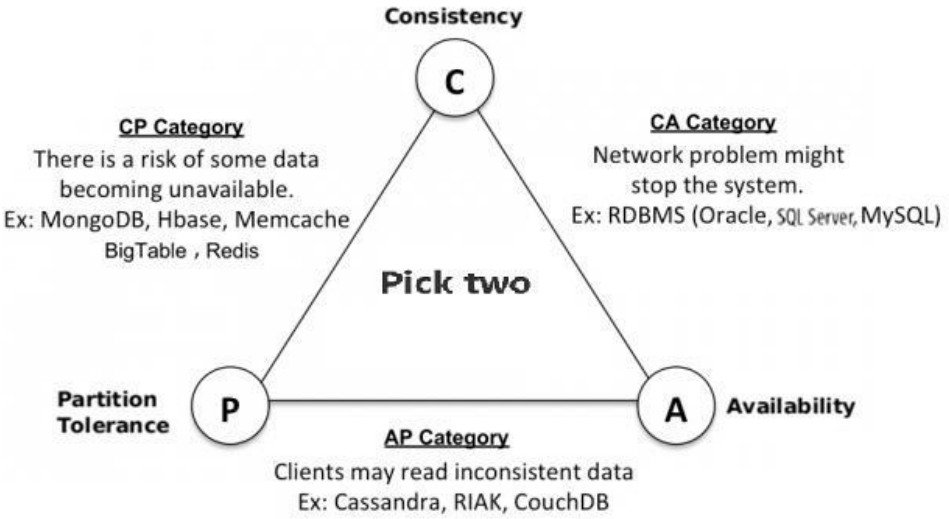
we're coming back to ACID and SQL

Consistency

- Many **applications** need **state replicated** across a wide area, for reliability, availability and low latency.



- **CAP Theorem:** *During a network partition, a system must choose between consistency and availability.*
  - Consistency
  - Availability
  - Partition tolerance (e.g., network partitioning)



Src: CAP Theorem by Eric Brewer (2000) , formally proved by Gilbert and Lynch in 2002

Why Consistency matters?

- Consider these examples:
  - **Example 1: People you do not want seeing your photos**
    - Alice removes mom from list of people who can view photos
    - Alice posts embarrassing photos from Spring Break
    - Can mom see Alice's photo?
  - **Example 2: Why am I still getting messages?**
    - Bob unsubscribes from the mailing list
    - Message sent to mailing list right after
    - Does Bob receive the message?
- Consistency is a complex topic, and we will get back to it when we discuss replication at scale.

Consistency models

- **Two main choices:**
  - **Strongly consistent** operations (e.g., use Paxos, Raft, etc.)
    - Often at the cost of additional latency for the common case
  - **Inconsistent** operations
    - Better performance / availability, but applications are harder to write and reason about the model

- **Many applications aiming for high availability gravitated towards eventual consistency**
  - Order of posts in **LinkedIn** news feed? Access from multiple devices?
  - When your post gets shown to friends in US or Asia?
  - Count of song popularity in **Spotify**?

- But, **modern data analytics (data lakes, training ML on PBs of data) require strong consistency**
  https://www.allthingsdistributed.com/2021/04/s3-strong-consistency.html

"Eventual Consistency Today: Limitations, Extensions, and Beyond" by Bailis and Ghodsi in ACM Queue vol. 11, issue 3, 2013

Document data model (2000s)

- A collection of record documents that contain a hierarchy of field/value pairs.
  - Each document field is identified by a name;
  - A field's value can be either a scalar type, an array of values, or another document.

```
{<field>: <scalar|[values]|{document}>}
```

  - Applications do not predefine schema.
  - Modern implementations use JSON. Older systems use XML or custom object representations.

```
{"name": "First Last",
 "orders": [{"id": 123, "items": [...]},
            {"id": 456, "items": [...]}]
}
```

Document DBMSs

- Storing data as documents removes the *impedance mismatch* between how application code interacts with data and how the system stores it.
  - Recall *object-relational mapping* between objects in application code and the relations in our RDBMS.
  - Document DBMSs offer *schema on read* (i.e., the structure of the data is implicit and only interpreted when the data is read) – useful when the items don't have the same structure.
  - *Dynamic data model* – allows uploading self-contained documents as a JSON
- *Denormalizing* entries into nested structures (i.e., pre-joining) avoids the need for joins or multiple queries to retrieve data related to an object.
  - Better locality as all *relevant information* is in one place and one query is *sufficient.*

Are document databases repeating history?

- In the **1970s** systems used a fairly simple data model called the ***hierarchical model***
  - It represented all data as a tree of records nested within records.
  - good for one-to-many relationships, but not for many-to-many relationships, and didn't support joins.
  - Developers had to decide to either duplicate data or manually resolve references between records
    - Similar to the issues reported by developers using document databases today!

- ***Network model*** or the CODASYL model
  - Generalization of the hierarchical model
    - Every record could have multiple parents – allowing for many-to-one and many-to-many relationships
    - The links between records were not foreign keys, but pointers like in a PL.
    - Resulting in labyrinth like nested structures, complicated access paths to reach data.

Document DBMSs – Discussion

- Great for certain use-cases like a messaging system or when dealing with multi-modal data.
- But the original premise of NoSQL (not-only-SQL) is diluted over time.
- Almost every major **NoSQL DBMS adjusted over time**:
  - Now almost every NoSQL DBMS added a SQL interface (proprietary SQL derived/inspired by SQL)
    - *e.g.*, DynamoDB's PartiQL, Cassandra CQL, Aerospike AQL, Couchbase SQL
  - Many NoSQL DBMSs also added strongly consistent (ACID) transactions.
  - Logical/physical data independence is back
- **RDBMSs** also evolved:
  - Introduced support for XML and later JSON (which was added in the SQL standard)
    - SQL:2016 added JSON operators, SQL:2023 added JSON types
  - allow "schema later" or schema inference, both becoming more prevalent in RDBMSs
  - Support querying multi-modal data and working with Data Lakes (and Lakehouses)

Column-family data model

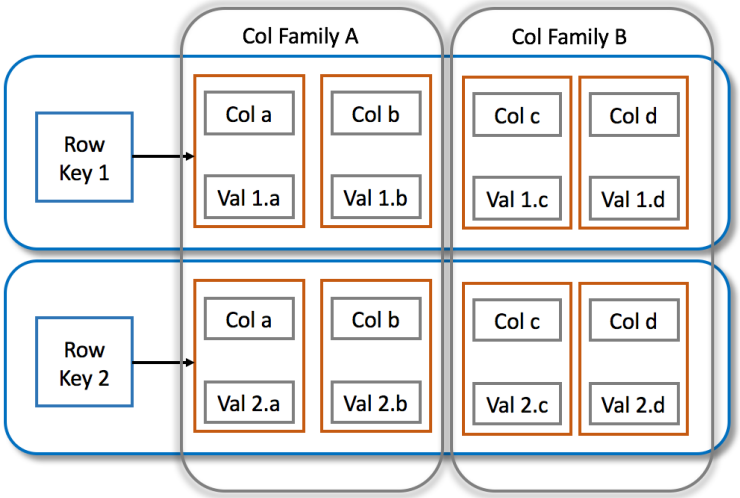
Column-Family data model (2000s)

- Still type of NoSQL databases
- Reduction of the document data model that only supports one level of nesting
  - *Row key* + *column families* – flexible schema per row
  - *Sparse storage* – columns are created only when needed
  - *Data locality* – grouped by access pattern.
- Another name is *wide-column* data model
- Typical syntax would be: `{<field>: <scalar|[values]>}`

Column-Family data model

- A column family database represents a map consisting of smaller maps:
  - Row key → column family → column qualifier → value (w/ optional timestamp)

- Think of a column-family as a table.
  - Each column must be assigned to a particular column-family and accessed via it.



https://www.dataversity.net/wp-content/uploads/2025/09/wcd-pic1.png

Column-family models

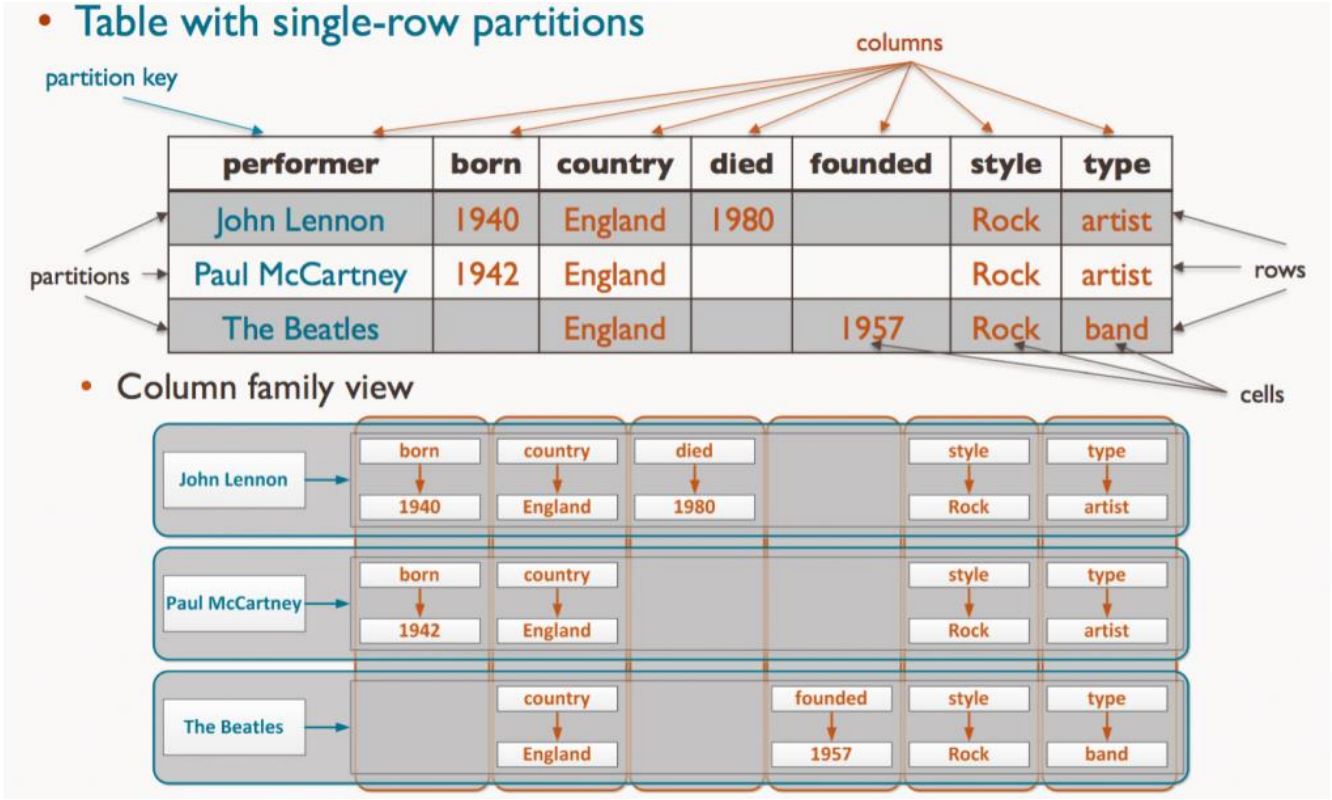
- Regarded as highly scalable, schema flexible alternative, optimized for high-throughput read/writes.

- Think of **column-family** as **flexible table** where each row can have different set of columns.
  - Unlike a traditional RDBMS, where all rows have the same structure (and potentially many NULLs)
  - Column families allow each row to only store the relevant data, reducing redundancy.

- It is *relation-like*, but each record can have *optional attributes*, and cells can contain an array of values
  - e.g., a mapping from user identifier keys to JSON documents that contain each user's profile info

```
User1000 → {"name": "Alice",
            "accounts": [123,456],
            "email": "xxx@yyy.edu"}
User1001 → {"name": "Bob",
            "email": ["yyy@zzz.edu", "zzz@www.edu"]}
```

Example



Src: https://studio3t.com/knowledge-base/articles/nosql-database-types/

Column-family DBMSs

- First implemented in Google's BigTable in 2004
  - **Wide-column model** with procedural API, no joins, no secondary indexes
  - Other notable systems: HBase, BigTable, ScyllaDB

- **Storage:**
  - Data is stored **column-family-first** (grouped by relevant fields)
- **Consistency:**
  - Like KV and document stores, writes are atomic.
  - But no multi-row ACID transactions are supported and are mainly AP systems (from CAP theorem).

- **Use-cases**
  - IoT data, time-series per key
  - Event logging, clickstreams
  - Write-heavy workloads

Column-family systems – Discussion

- **Wide-column ≠ columnar**
  - Wide-column → OLTP, distributed, row-partitioned, groups related data (families)
  - Columnar storage → OLAP, vectorized scan, entire tables are column-major

- As expected, **consistency** and **SQL** turned out to be **important** for many use-cases so:
  - Google initially built Spanner on top of BigTable, but later rewrote Spanner.
  - Cassandra replaced their Thrift-API with CQL
  - HBase now recommends using the Phoenix SQL front-end.

Graph model

What's a graph?

- A graph consists of two types of objects:
  - vertices (also known as nodes or entities), and
  - edges (also known as relationships or arcs)
- G = (V, E)
- A set of vertices (V, nodes) and edges (E)
- Many variants:
  - Directed, multi-graph, connected, cyclic, hyper-graph
- Add properties (key-value pairs) to vertices and/or edges to make them useful for data engineering.

Graph model

- Many kinds of data can be modeled as a graph:
  - *Social graph:* vertices are people, and edges indicate which people know each other
  - *The web graph:* vertices are web-pages, and edges indicate HTML links to other pages
  - *Road or rail networks:* vertices are junctions, and edges represent the roads or railway lines

- Various algorithms can then operate on these graphs:
  - *e.g.*, car navigation system can search the shortest path between two points on the road network
  - *e.g.*, PageRank can be used on the web graph to determine the popularity of a web-page.

Graph model

- Many applications use *knowledge graphs* to model semi-structured information.
  - Social media applications inherently contain graph-oriented relationships (“likes”, “friend of”).
- Directed multi-graph structure that supports *key/value labels* for nodes and edges
- Two most prevalent approaches to represent graphs are
  - *Property Graph*
  - *Resource Description Framework (RDF)*

Property Graph model

- **Each vertex consists of:**
  - A unique identifier
  - A set of outgoing edges
  - A set of incoming edges
  - A collection of properties (KV pairs)

- **Each edge consists of:**
  - A unique identifier
  - The vertex at which the edge starts
  - The vertex at which the edge ends
  - A label to describe the type of relationship between the vertices
  - A collection of properties (KV pairs)

- By using **different labels** for **different kinds of relationships**, you can store several **different kinds of information in a single graph** (while maintaining a clean data model)

Property Graph model

- PG DBMS maintains a directed multi-graph structure that supports key/value labels for nodes and edges.

- Nodes and relationships are *first-class citizens.*
  - it is compact, expressive and queryable by structure and properties.

- It is like an **object graph** where you can directly store attributes and relationships and traverse them.
  - Local IDs
  - informal semantics (no formal logic supporting it) → no reasoning,
  - but efficient for graph traversal, analytics and applications.

- **The Cypher Query Language:**
  - Declarative query language for property graphs, created for the **Neo4j** database.

```
(Alice:Person {age:29})-[:KNOWS {since: 2020}]-> (Bob:Person {age:31})
```

Resource Description Framework (RDF)

- **RDF DBMSs (aka triplestores)** – model a directed graph with labeled edges.
  - Comes from the semantic web / AI reasoning world.
  - **Data** = set of **triples** (subject – predicate – object).
  - Each **triple** is a fact:
    - *e.g.*, <Alice> <knows> <Bob>
  - All **entities** and **predicates** are URIs (global identifiers)
  - The **object** is either a resource (URI) or a literal.

Subject	Predicate	Object
Alice	knows	Bob
Alice	type	Person
Alice	age	29

  - Adding “properties” means adding more triples in one giant table of triples.

- It is like a **fact table** of statements that machines can reason about.

The SPARQL query language

- SPARQL is a query language for triplestores that use the RDF data model.
  - SPARQL Protocol and RDF Query Language (pronounced sparkle)

- Comparison to Cypher
  - Predicate search:

  - `(person)-[:BORN_IN]->()-[:WITHIN*0..]->(location) # cypher`
  - `?person :bornIn /:within* ?location.  # SPARQL`

- Because RDF does not distinguish between properties and edges but just uses predicates for both, you can use the same syntax for matching properties.

```
SELECT ?person ?location
WHERE {
   ?person :bornIn /:within* ?location.
}
```

"Find all ?person where their :bornIn value is connected to ?location by zero or more :within relationships."

Graph stores and Graph DBMSs

- **Graph** is the data model.
- **Graph databases** are systems that make graph *queryable and mutable.*
- **Graph stores** (analytic engines) make the graphs *computationally analyzable at scale.*

Two main categories

- **Transactional (OLTP workloads)**
  - *e.g.*, Neo4j is the most popular graph DBMS for OLTP applications.
    - Supports edges using pointers, but does not cluster nodes with their parent or offspring.
    - Good for traversing long chains – pointer chasing (RDBMS needs to use joins w/ recursive CTEs)

- **Analytics (OLAP workloads)** – derive information from the graph
  - *e.g.*, TigerGraph, Giraph, Graphlab (computing fabric)
  - Algorithm choice and data representation are key to performance.

Discussion

- **Document databases** target use-cases where data comes in self-contained documents and relationships between one document and another are rare.
- **Graph databases** in contrast target use-cases where anything is potentially related to everything.
- Document and graph databases do not enforce a schema, and come with own query languages.
- Models can be *emulated* in other models. *e.g.*, graph data can be represented in a relational database
  - Node (node_id, node_data)
  - Edge (source_id, target_id, edge_data)
- SQL:2023 introduced **SQL/PGQ** – for defining and traversing graphs in a RDBMS.
  - Based on Neo4j Cypher, Oracle PGQL, TigerGraph GSQL (ISO standard as of 2024)

Array model

Array model

- There are many areas of computing where **arrays** are an obvious data representation.
  - **Vectors** (one dimensional), **matrices** (two dimensions), and **tensors** (three or more)
  - *e.g.,* Scientific surveys for geographic regions usually represent data as a multi-dimensional array that stores sensor measurement using location / time-based coordinates.

```
(dimension1, dimension2, ..., [values])
```

```
(latitude, longitude, time, [vector of values])
```

  - Several other datasets look like that: genomic sequencing, computational fluid dynamics, etc.
    - *e.g.*, Climate modelling: **4D/5D arrays**
    - *e.g.,* Satellite imaging: **2D/3D arrays** (+irregularities)
    - *e.g.,* Genome: long string **arrays,** or **3D arrays** (sample name, chromosome, position).
    - Arrays are also the core of most ML datasets.

Array data model

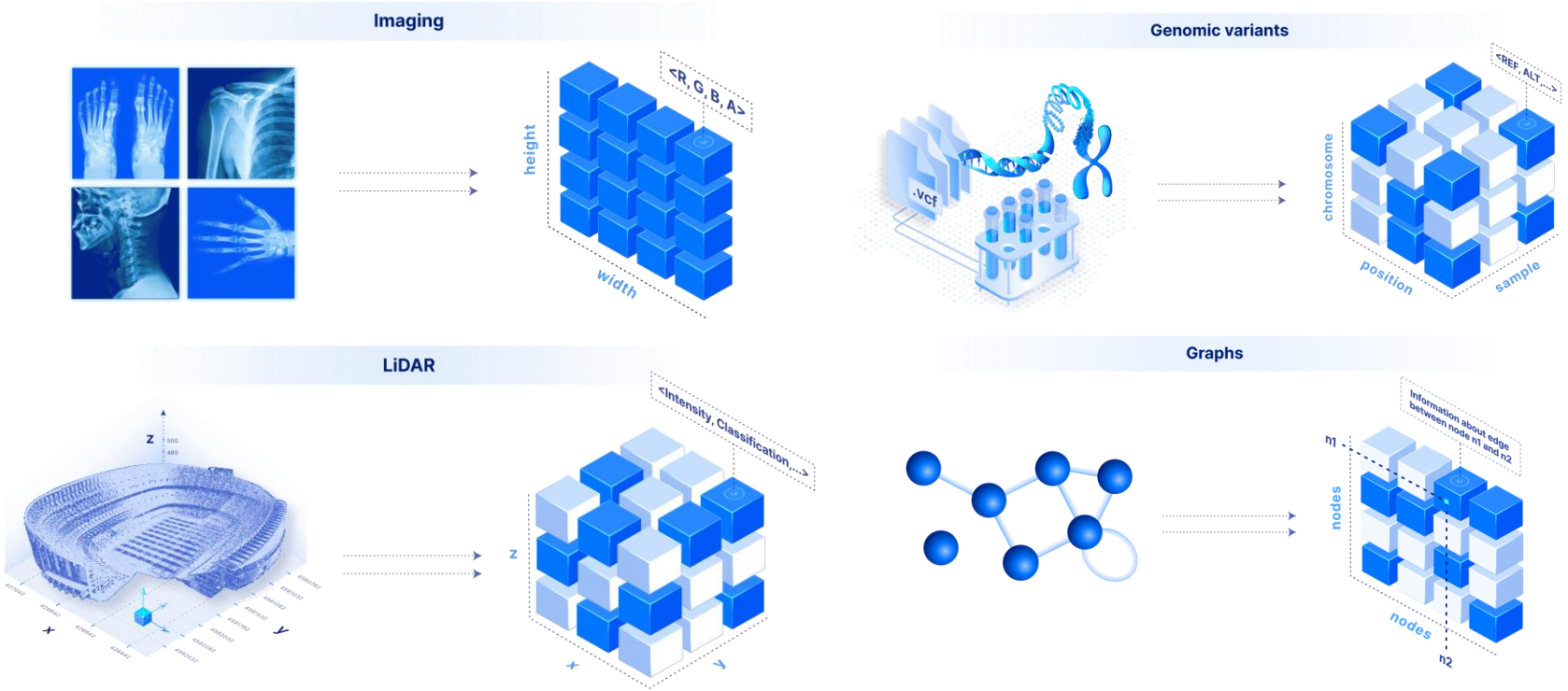
- Distinction between **dense** and **sparse** arrays.

  - **Dense** – any number of dimensions.
    - each **dimension** must have a **domain** with `int` values, and all dimensions have the same data type.
    - an **array element** is defined by a unique set of dimension coordinates and is called a **cell.**
    - A logical **cell** can store more than one value of potentially different types (`int`, `float`, `string`, etc.)
    - An **attribute** defines a collection of values that have the same type across all cells.

  - **Sparse** – similar to a dense array, with three key differences
    - **Cells** in a sparse array can be **empty**
    - The dimensions can have **heterogeneous types** including floats and strings.
    - **Cell multiplicities** *(i.e.,* cells with the same dimension coordinates) are allowed.

- Which one to use depends on the application and can greatly affect performance.

Examples



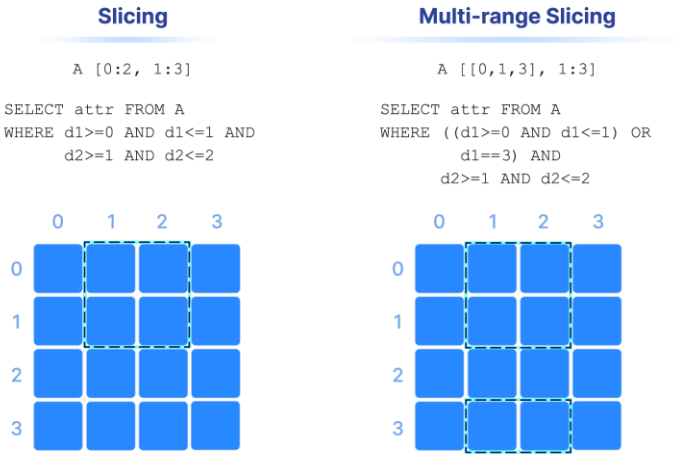
Src: https://www.tiledb.com/blog/why-arrays-as-a-universal-data-model

Array DBMSs

- Most notable **Array DBMSs**:
  - Rasdaman, kdb+, SciDB, TileDB.

- Array systems are optimized for rapidly performing **range conditions on dimension coordinates.**

- Range conditions on dimensions are often called **slicing** and the results are a **slice** or **subarray.**



Src: https://www.tiledb.com/blog/why-arrays-as-a-universal-data-model

Array DBMS

- **System challenges:**
  - Array data does not always align to a regular integer grid
    - *e.g.*, a geo-spatial data is split into irregular shapes
    - An application can then map such grids to integer coordinates with metadata that describes this mapping.
  - Unlike row- or column- RDBMSs, querying array data in arbitrary dimensions is difficult.
    - E.g., how do we store a multi-dimensional array on a linear physical storage medium.
      - Special indexes and data structures to support efficient traversal across the array dimensions.
- **SQL:2019 added multi-dimensional arrays (SQL/MDA)**
  - Allows SQL to represent arrays with arbitrary dimensions using integer-based coordinates.
  - Array data access pattern necessitates a dedicated execution engine.

Vector data model

- One-dimensional arrays are used for nearest-neighbor search (exact or approximate)
  - Used for semantic search on **embeddings** generated by ML-trained transformer models
  - Native integration with modern ML tools and APIs (e.g., LangChain, OpenAI)

- At their core, these systems use **specialized indexes** to perform nearest neighbor searches quickly.
  - We will discuss this in more detail next Tuesday!

Which model?

- Scenarios:
  Pick a model for each:

  - Session store for a web app
  - Product catalog with varying attributes
  - Fraud detection over money transfers
  - Sensor readings from 10’000 IoT devices
  - Satellite imagery tiles
  - Find documents similar to this one

Comparative table

Model	Schema	Query style	Scaling	Strength	System
Relational	Fixed	SQL	Vertical	Consistency	Oracle, Spanner
Key-Value	None	GET/PUT	Horizontal	Simplicity	Redis
Document	Semi-str.	Query by field	Horizontal	Flexibility	MongoDB
Wide-column	Dynamic families	CQL	Horizontal	High write throughput	Cassandra
Graph	schema-optional	Traversal	Depends	Relationship analysis	Neo4j
Array / TS	Ordered time axis	Range queries	Horizontal	Time analysis	TileDB/SciDB
Vector	High-dimensional	Similarity	Horizontal	Semantic search	Milvus, Pinecone

References

Book:

- “Designing Data-Intensive Applications” by Martin Kleppmann
  - Chapter: Data models and query languages

- Foundations in Data Engineering – NoSQL Databases / Other Data Models by Prof. Neumann (TUM)
- What goes around, comes around and around – paper by Andy Pavlo and Michael Stonebraker